

Rapport de projet DAAR 1

automate. Clone de egrep avec support partiel des ERE

Younes MANSOUS 21406000

Moustafa FELLAGUE 21401043

Master 2 STL – Sorbonne Université

Octobre 2026

Table des matières

1 Objectif du projet **2**

2 Structures de données et architecture **2**
2.1 Arbre syntaxique abstrait (AST) 2
2.2 Automates et représentation des transitions 3
2.3 Architecture générale du programme 3

3 Algorithmes implémentés **4**
3.1 Construction de l'AFN-ε 4
3.2 Déterminisation de l'AFN 4
3.3 Minimisation de l'AFD 5
3.4 Algorithme KMP 5
3.5 Recherche naïve et recherche en une passe 5

4 Analyse de complexité et choix d'implémentation **6**

5 Expérimentations **6**
5.1 Présentation des données de test 6
5.2 Tests de correction 6
5.3 Protocole de mesure des performances 6

6 Résultats et discussion **7**
6.1 Comparaison des algorithmes 7
6.2 Étude d'un cas défavorable 7
6.3 Effet de la minimisation 7
6.4 Fréquence des motifs 9

7 Conclusion et perspectives **9**

1 Objectif du projet

L'objectif de ce projet est de développer en Java un programme de recherche de motifs dans des fichiers textuels, inspiré de la commande Unix `egrep`. Notre programme doit permettre de rechercher les lignes contenant une expression régulière, en prenant en charge une partie des expressions régulières étendues (ERE).

Pour cela, nous avons implémenté plusieurs algorithmes étudiés en cours. Une expression régulière est d'abord analysée afin de construire un arbre syntaxique abstrait (AST). Cet arbre est ensuite transformé en automate fini non déterministe avec transitions epsilon (AFN-ε), à l'aide de la construction de Thompson. L'automate obtenu est déterminisé, puis minimisé avec l'algorithme de Moore.

Nous avons également implémenté l'algorithme de Knuth-Morris-Pratt (KMP), utilisé pour rechercher des motifs constitués uniquement de caractères littéraux. Pour la recherche avec les automates, deux méthodes ont été développées : une recherche naïve, qui essaie chaque position du texte, et une recherche en une seule passe, basée sur la transformation de l'expression régulière R en $.*R$.

Enfin, nous cherchons à vérifier la correction de nos implémentations et à comparer leurs performances sur des fichiers textuels, notamment des ouvrages issus du projet Gutenberg. Les expériences permettront d'étudier l'influence de la taille des textes, du type de motif et du choix de l'algorithme sur les temps d'exécution.

2 Structures de données et architecture

2.1 Arbre syntaxique abstrait (AST)

La première étape pour transformer une expression régulière en automate consiste à construire un arbre syntaxique abstrait (AST). Nous avons donc décidé de représenter chaque élément d'une expression régulière par un nœud dans cet arbre.

Notre implémentation prend en charge les principaux opérateurs demandés dans l'énoncé :

— Les caractères littéraux, comme `a` ou `b`.

— La concaténation, par exemple `ab`.

— L'alternative (`|`), par exemple `a|b`.

— La répétition de Kleene (`*`), par exemple `a*`.

— Le joker (`.`), qui représente un caractère quelconque.

— Le mot vide (ε).

— Les parenthèses, qui permettent de regrouper les expressions et de définir les priorités.

Pour représenter ces éléments, nous avons créé plusieurs classes : `AstChar`, `AstConcat`, `AstOr`, `AstRepeat`, `AstDot` et `AstVide`. Toutes ces classes implémentent l'interface `AstI`, qui permet de manipuler les différents nœuds de manière uniforme.

Pour organiser cette partie, nous nous sommes inspirés de ce que nous avions appris dans le module DLP de l'année précédente, notamment de l'utilisation des design patterns *Factory* et *Visitor*.

Le pattern *Factory*, implémenté à travers la classe `AstFactory`, permet de centraliser la création des différents nœuds de l'arbre, sans avoir à instancier directement chaque classe.

Le pattern *Visitor*, quant à lui, permet de parcourir l'AST et d'effectuer différentes opérations sur ses nœuds sans modifier leur structure. Nous avons notamment utilisé ce principe avec l'interface `AstVisitor` pour transformer notre arbre syntaxique en automate fini non déterministe, à travers la classe `AstToAfnVisitor`.

Enfin, nous avons développé un analyseur syntaxique dans la classe `Parser`, qui lit l'expression régulière et construit l'AST correspondant, en respectant les priorités des opérateurs et les parenthèses.

2.2 Automates et représentation des transitions

Après avoir construit l'arbre syntaxique, nous avons dû choisir comment représenter les automates et leurs transitions en Java. Nous avons donc créé plusieurs classes pour organiser cette partie du projet.

Tout d'abord, nous avons créé la classe `Etat`, qui représente un état d'un automate, ainsi que la classe `Transition`, qui permet de représenter une transition entre deux états, associée à un symbole.

Nous avons ensuite créé les classes `Afn` et `Afd`, qui représentent respectivement les automates finis non déterministes et déterministes. Ces deux classes implémentent une interface commune, `AutomateI`, afin de regrouper les opérations communes aux automates.

Au début, nous avions choisi de stocker les transitions dans de simples listes (`List<Transition>`). Cependant, nous avons remarqué que cette représentation nécessitait de parcourir les transitions pour retrouver celles correspondant à un état et à un symbole donnés.

Nous avons donc décidé de remplacer cette représentation par des `HashMap`, afin d'accéder plus rapidement aux transitions et d'améliorer les performances de nos algorithmes.

Dans le cas de l'AFN, nous utilisons la structure suivante :

```
Map<Etat, Map<String, Set<Etat»>
```

Cette structure permet d'associer plusieurs états d'arrivée à un même état de départ et à un même symbole. Elle permet également de représenter les transitions epsilon (ε), qui ne consomment aucun caractère.

Pour l'AFD, nous avons utilisé une structure similaire, mais plus simple :

```
Map<Etat, Map<String, Etat»
```

En effet, contrairement à l'AFN, un AFD ne peut avoir qu'un seul état d'arrivée pour un état donné et un symbole donné.

Avec cette représentation, la recherche d'une transition se fait en temps constant en moyenne, grâce aux `HashMap`, contrairement à notre première version utilisant des listes, où la recherche pouvait nécessiter un parcours linéaire des transitions.

Chaque automate conserve également une liste de ses états, un état initial et un ensemble d'états finaux.

Enfin, pour gérer le joker `.`, qui représente un caractère quelconque, nous avons prévu un symbole particulier `AUTRE` dans notre AFD. Il permet de traiter les caractères qui ne font pas explicitement partie de l'alphabet de l'expression régulière.

Ces structures nous ont permis de manipuler les automates plus efficacement, notamment lors de la déterminisation, de la minimisation et de la recherche de motifs dans les fichiers textuels.

2.3 Architecture générale du programme

Nous avons organisé notre programme en plusieurs packages Java afin de séparer les différentes étapes du traitement d'une expression régulière.

Le package `parser` contient la classe `Parser`, qui analyse la RegEx et construit l'arbre syntaxique représenté par les classes du package `ast`.

Le package `automate` regroupe les classes nécessaires à la construction et à la transformation des automates (AFN, AFD, déterminisation et minimisation). La classe `ConstructionAutomate` permet d'enchaîner ces différentes étapes.

Nous avons également créé le package `experimentation`, qui contient les méthodes de recherche de motifs, notamment `MatcherNaif` et `MatcherUnePasse`, ainsi que la lecture des fichiers textuels. Le package `kmp` contient notre implémentation de l'algorithme KMP.

Enfin, la classe `Main` sert de point d'entrée au programme. Elle permet de saisir une expression régulière, de choisir un fichier et un algorithme de recherche. Par défaut, le programme

sélectionne KMP pour les motifs littéraux et la recherche en une passe pour les autres expressions.

Cette organisation nous a permis de séparer les différentes parties du programme et de tester plus facilement chaque algorithme.

3 Algorithmes implémentés

3.1 Construction de l'AFN-ε

Pour transformer notre AST en automate, nous avons choisi d'utiliser l'algorithme de Thompson, proposé en 1968 [1].

Le principe est assez simple : on construit d'abord de petits automates pour chaque élément de l'expression régulière, puis on les relie avec des transitions epsilon (ε), qui permettent de changer d'état sans lire de caractère.

La construction suit quelques règles :

— **Caractère et joker :** on crée deux états reliés par une transition.

— **Mot vide :** on crée un état à la fois initial et final.

— **Concaténation :** on relie les deux automates par une transition epsilon.

— **Alternative :** on ajoute des transitions epsilon pour choisir entre deux automates.

— **Répétition (*) :** on utilise des transitions epsilon pour répéter une expression zéro ou plusieurs fois.

Nous avons implémenté cet algorithme dans la classe `AstToAfnVisitor`, en utilisant le pattern Visitor. Chaque type de nœud de l'AST possède une méthode qui permet de construire l'automate correspondant.

L'avantage de Thompson est qu'il est assez simple à implémenter et que sa complexité théorique est linéaire, soit $O(m)$ pour une expression de taille m. Cependant, il peut produire beaucoup de transitions epsilon et plusieurs transitions possibles pour un même symbole.

C'est pourquoi nous avons ensuite choisi de déterminiser l'AFN obtenu, afin de faciliter la recherche de motifs.

3.2 Déterminisation de l'AFN

Après la construction de Thompson, nous obtenons un AFN qui peut avoir plusieurs transitions possibles pour un même symbole. Nous avons donc utilisé l'algorithme de déterminisation par sous-ensembles, présenté notamment dans les travaux de Hopcroft et Ullman [2], pour le transformer en AFD.

Le principe est de regrouper plusieurs états de l'AFN dans un seul état de l'AFD, afin d'avoir au maximum une transition par symbole.

Pour cela, nous utilisons deux opérations :

— **Epsilon-fermeture :** récupérer tous les états accessibles sans lire de caractère, grâce aux transitions epsilon.

— **Move :** récupérer les états accessibles en lisant un symbole donné.

Nous commençons par l'epsilon-fermeture de l'état initial, puis nous calculons les transitions pour chaque symbole. Chaque nouvel ensemble d'états rencontré devient un état de l'AFD. On répète cette opération jusqu'à ce qu'il n'y ait plus de nouveaux états à traiter.

Dans notre projet, nous avons implémenté cet algorithme dans la classe `Determinisation`, avec les méthodes `epsilonClosure`, `move` et `convertir`. Nous avons utilisé des `Set` pour regrouper les états et une file pour traiter progressivement les nouveaux états. Nous avons également adapté le traitement des transitions pour prendre en compte le joker `.` et le symbole `AUTRE`.

L'avantage de cette méthode est qu'elle simplifie la recherche, puisqu'un AFD ne possède qu'une seule transition possible par symbole et par état. Cependant, elle peut produire beaucoup d'états : un AFN de n états peut théoriquement donner un AFD de 2^n états dans le pire cas.

Pour limiter ce problème, nous avons choisi de construire uniquement les états accessibles depuis l'état initial. Cela ne supprime pas le risque d'avoir beaucoup d'états, mais évite d'en créer inutilement.

3.3 Minimisation de l'AFD

Après la déterminisation, nous obtenons un AFD qui peut contenir beaucoup d'états, dont certains sont équivalents. Nous avons donc choisi d'utiliser l'algorithme de Moore pour réduire le nombre d'états tout en conservant le même langage reconnu.

Le principe est de regrouper les états qui ont le même comportement. Pour cela, on sépare d'abord les états finaux des états non finaux. Ensuite, on divise progressivement ces groupes selon les transitions de chaque état, jusqu'à ce qu'aucune séparation supplémentaire ne soit nécessaire.

Nous avons implémenté cet algorithme dans la classe `Minimisation`. Nous commençons par supprimer les états inaccessibles, puis nous ajoutons si nécessaire un état puits pour compléter les transitions. Nous appliquons ensuite l'algorithme de Moore pour regrouper les états équivalents.

Nous avons également choisi de retirer les états qui ne permettent plus d'atteindre un état final, afin d'éviter de conserver des transitions inutiles lors de la recherche.

L'avantage de cette méthode est de réduire la taille de l'automate et de limiter la mémoire utilisée. En revanche, elle nécessite un traitement supplémentaire avant la recherche. Sa complexité dans le pire cas est en $O(|\Sigma|n^2)$, avec n le nombre d'états et Σ l'alphabet.

Nous avons retenu cette méthode car la minimisation n'est effectuée qu'une seule fois lors de la construction de l'automate, qui peut ensuite être réutilisé pour rechercher le motif dans plusieurs textes.

3.4 Algorithme KMP

En complément des automates, nous avons implémenté l'algorithme de Knuth-Morris-Pratt (KMP), qui permet de rechercher un motif littéral dans un texte, c'est-à-dire un motif sans opérateurs d'expression régulière.

Le principe de KMP est d'éviter de comparer plusieurs fois les mêmes caractères lorsqu'une correspondance échoue. Pour cela, il construit d'abord une table des préfixes, qui indique à quelle position reprendre la recherche après un échec.

Nous avons implémenté cet algorithme dans la classe `Kmp`, avec une méthode `calculerEchec` pour construire cette table. Nous utilisons également la méthode `motifLitteral` pour vérifier si l'expression peut être traitée par KMP.

L'avantage de KMP est sa complexité linéaire en $O(n+m)$, où n représente la taille du texte et m celle du motif. Contrairement à une recherche classique qui peut revenir en arrière, KMP évite les comparaisons inutiles.

Cependant, cet algorithme ne prend pas en charge directement les opérateurs des expressions régulières comme `*` ou `|`. Nous avons donc choisi de l'utiliser uniquement pour les motifs littéraux.

3.5 Recherche naïve et recherche en une passe

Pour rechercher une expression régulière dans un texte à partir de notre AFD, nous avons implémenté deux méthodes différentes.

La première, dans la classe `MatcherNaif`, consiste à essayer de reconnaître le motif à partir de chaque position du texte. Si la recherche échoue, on recommence à la position suivante. Cette méthode est simple à comprendre et à implémenter, mais elle peut parcourir plusieurs fois les mêmes caractères. Sa complexité peut donc atteindre $O(n^2)$ dans le pire cas, pour un AFD déjà construit.

La deuxième méthode, implémentée dans `MatcherUnePasse`, permet d'éviter ces répétitions. Pour cela, nous transformons l'expression régulière R en $.*R$, afin de pouvoir reconnaître le motif même s'il ne commence pas au début du texte.

Nous construisons ensuite un AFD à partir de cette nouvelle expression, puis nous parcourons chaque ligne une seule fois. Dès qu'un état final est atteint, nous savons que le motif a été trouvé.

Cette approche permet d'obtenir une recherche en $O(n)$, une fois l'automate construit. En revanche, l'ajout de $.*$ peut rendre la construction de l'automate plus coûteuse.

Nous avons conservé les deux méthodes afin de comparer leurs performances et de montrer l'intérêt d'une recherche en une seule passe, notamment sur les fichiers textuels de grande taille.

4 Analyse de complexité et choix d'implémentation

Nous avons effectué des recherches sur les complexités théoriques des différents algorithmes, notamment à partir des notions vues en cours et de la littérature existante, afin de mieux comprendre leurs avantages et leurs limites.

Ces recherches nous ont aidés à faire certains choix d'implémentation, comme le remplacement des listes de transitions par des `HashMap`, pour rendre leur accès plus rapide.

Nous avons également choisi de minimiser les automates afin de réduire leur nombre d'états, et d'utiliser KMP pour les motifs littéraux, puisqu'il permet une recherche en temps linéaire.

Enfin, nous avons conservé deux méthodes de recherche avec les automates pour pouvoir comparer leurs performances : une méthode naïve, qui peut atteindre $O(n^2)$ dans le pire cas, et une méthode en une passe, de complexité $O(n)$ une fois l'automate construit.

Les tests de performance présentés dans la suite du rapport nous ont permis de comparer ces résultats théoriques avec les temps d'exécution obtenus en pratique.

5 Expérimentations

Pour la mise en place des tests, nous nous sommes également aidés de ChatGPT 6 Astra [3], notamment pour la création des scripts de test et de leur configuration.`Main`.

5.1 Présentation des données de test

Pour évaluer notre programme, nous avons utilisé neuf fichiers textuels (`text1.txt` à `text9.txt`), ainsi que cinq fichiers artificiels permettant de tester des cas particuliers.

Nous avons également préparé six fichiers d'entrée contenant 34 expressions régulières, réparties entre les motifs littéraux, les alternatives, les répétitions, les jokers et les cas particuliers comme le mot vide et les caractères accentués.

Les textes ont des tailles différentes, ce qui nous permet d'observer l'évolution des performances selon la quantité de données à parcourir.

5.2 Tests de correction

Pour vérifier notre implémentation, nous avons comparé les résultats des algorithmes avec ceux de `java.util.regex.Pattern.find()`.

Au total, nous avons réalisé 8 007 480 comparaisons, sans aucune erreur détectée.

Nous avons également comparé les résultats des deux méthodes utilisant les automates sur 3 490 440 cas. Aucune divergence n'a été observée.

Ces résultats montrent que les algorithmes donnent des résultats cohérents sur l'ensemble des expressions et des fichiers testés.

5.3 Protocole de mesure des performances

Nous avons mesuré les temps d'exécution avec `System.nanoTime()`.

Pour chaque configuration, nous avons effectué 3 exécutions de chauffe, suivies de 10 mesures. Nous avons ensuite calculé la moyenne et l'écart-type.

Le temps de lecture des fichiers est exclu des mesures. Nous avons également mesuré séparément le temps de construction des automates.

Les performances ont été comparées sur différents types de motifs et plusieurs tailles de fichiers.

6 Résultats et discussion

6.1 Comparaison des algorithmes

La figure 1 présente les temps de recherche du motif littéral `love`.

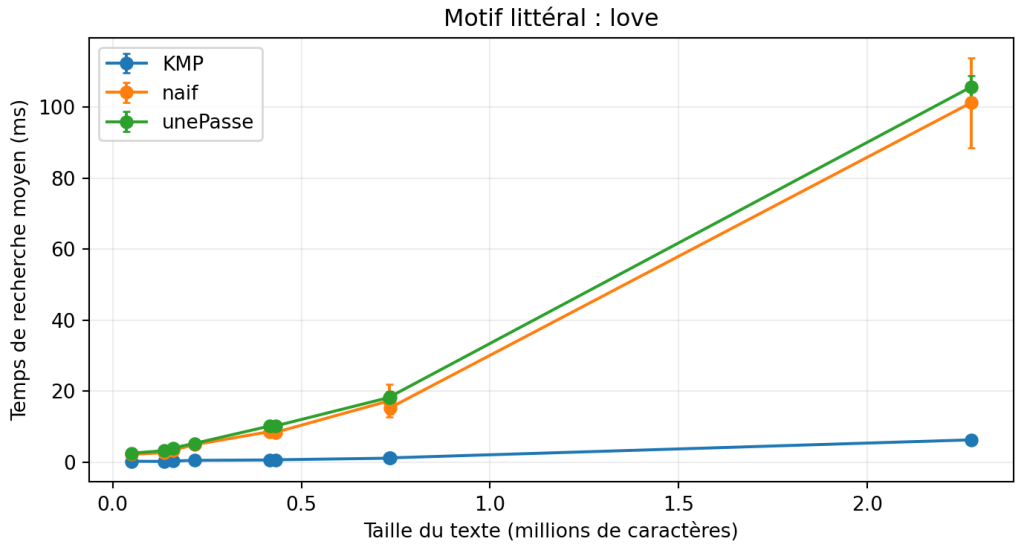


FIGURE 1 – Comparaison des performances pour le motif love.

Nous remarquons que KMP est nettement plus rapide que les deux méthodes utilisant les automates. Cela s'explique notamment par le fait que KMP est spécialement conçu pour la recherche de chaînes de caractères.

Pour les expressions régulières plus complexes, comme `a.*b`, la recherche en une passe est généralement plus rapide que la recherche naïve.

6.2 Étude d'un cas défavorable

Nous avons également testé le motif `a.*b` sur des lignes contenant uniquement des caractères `a`, avec des tailles croissantes.

À 20 000 caractères, la recherche naïve prend en moyenne 3 537 ms, contre seulement 0,327 ms pour la recherche en une passe.

Cette différence s'explique par le fait que la méthode naïve recommence la recherche à chaque position, alors que la seconde ne parcourt le texte qu'une seule fois.

Ces résultats illustrent bien la différence entre les comportements quadratique et linéaire dans ce cas particulier.

6.3 Effet de la minimisation

On as aussi comparé le nombre d'états avant et après les différentes transformations et nous avons constater que la déterminisation et la minimisation permettent souvent de réduire le nombre d'états. Cependant, pour certaines expressions simples, la minimisation n'apporte aucune réduction supplémentaire.

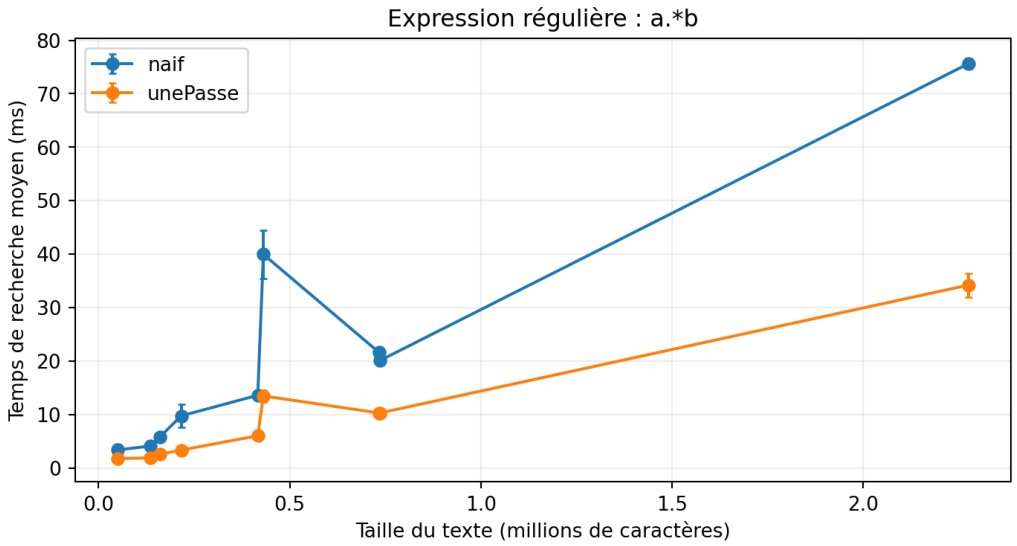


FIGURE 2 – Comparaison des performances pour le motif a.*b.

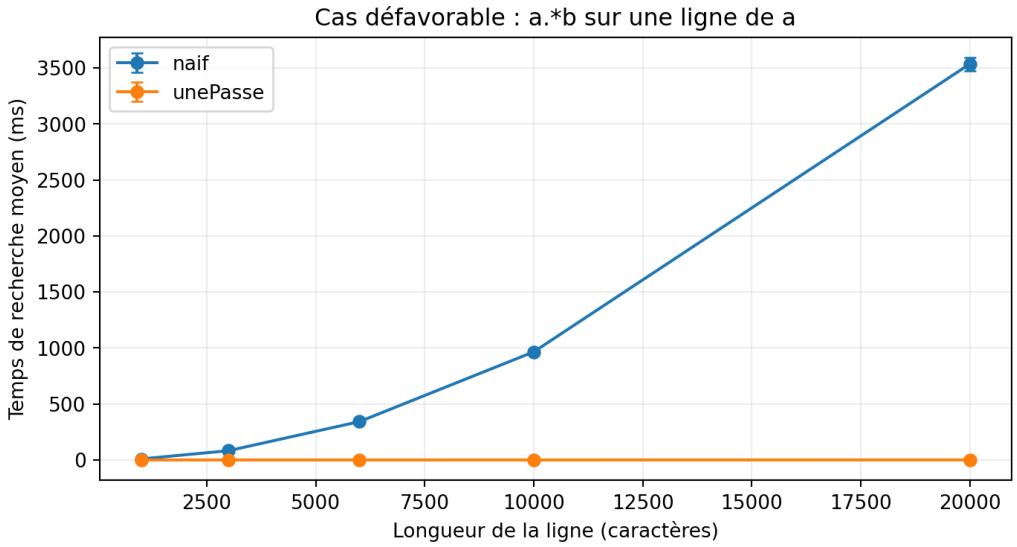


FIGURE 3 – Temps de recherche sur un cas défavorable.

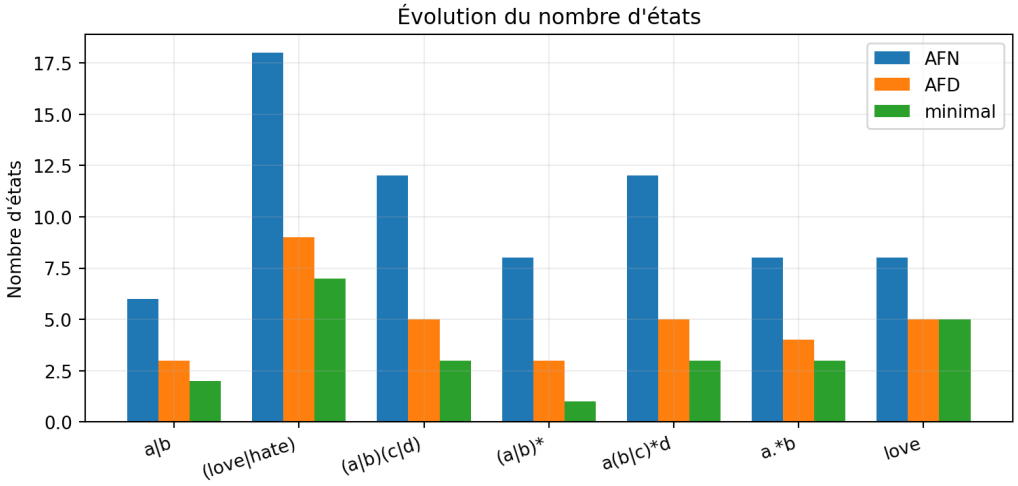


FIGURE 4 – Nombre d'états des automates selon les expressions.

6.4 Fréquence des motifs

Enfin, nous avons étudié le nombre de lignes contenant le motif `love` dans les différents fichiers textuels.

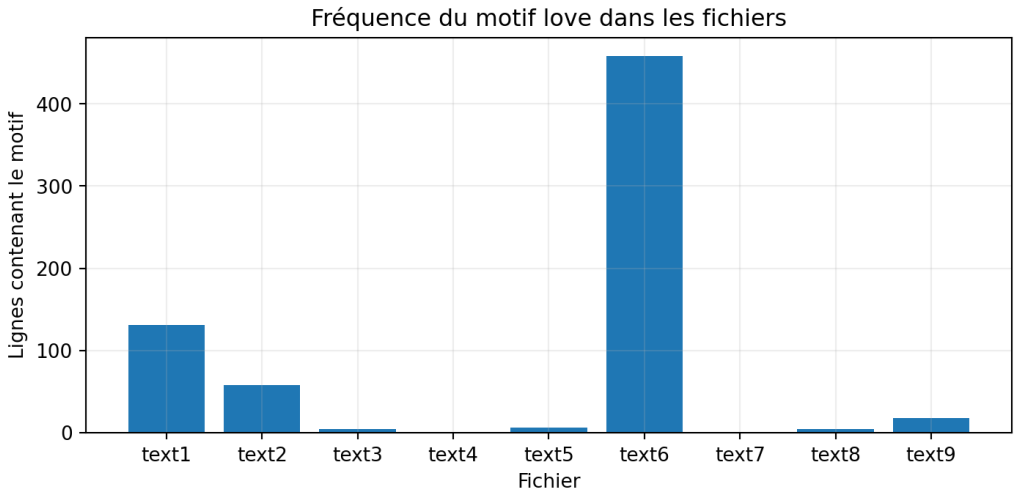


FIGURE 5 – Nombre de lignes contenant le motif love.

La fréquence d'apparition d'un motif varie selon le fichier. Ce facteur peut également influencer les temps de recherche, puisqu'un algorithme peut s'arrêter dès qu'il trouve une correspondance sur une ligne.

7 Conclusion et perspectives

Ce projet nous a permis de mieux comprendre le fonctionnement des expressions régulières et les différents algorithmes utilisés pour rechercher des motifs dans un texte.

Nous avons réussi à développer un clone simplifié de `egrep` en Java, en utilisant les automates et l'algorithme KMP. Nous avons également pu mettre en pratique plusieurs notions vues en cours sur les automates.

Les tests de correction n'ont révélé aucune erreur sur les cas étudiés. Les tests de performance ont également montré que KMP est particulièrement efficace pour les motifs littéraux, tandis que la recherche en une passe permet d'éviter les ralentissements importants de la méthode naïve, notamment dans les cas défavorables.

Mais notre programme reste limité à une partie des expressions régulières étendues (ERE). Pour aller plus loin, nous pourrions ajouter de nouveaux opérateurs, améliorer la gestion des caractères Unicode et optimiser davantage la construction des automates.

Ce projet nous a donc permis de passer de la théorie à une implémentation concrète, mais aussi de constater que le choix de l'algorithme peut avoir un impact important sur les performances.

Références

[1] Ken Thompson, *Regular Expression Search Algorithm*, Communications of the ACM, vol. 11, no. 6, pp. 419–422, 1968.

[2] John E. Hopcroft, Rajeev Motwani et Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3e édition, Pearson, 2006.

[3] OpenAI, *ChatGPT 6 Astra* & Anthropics, *Claude code*, outils d'intelligences artificielles utilisés comme aide à l'implémentation de la classe `Main`, ainsi qu'à la création des scripts et à la configuration des tests, octobre 2026.